

Part 4: Vibe-Coded Apps – The New Spreadsheets and Why Security Must Adapt

Inefficiencies as an Economic Outcome, Not “Stupidity”

In large enterprises, many of the persistent inefficiencies are better understood as economic and structural issues rather than simple ignorance or incompetence. People are not *stupid*; they respond to incentives and constraints. Organizations often **optimize for immediate delivery over long-term productivity**, creating environments where each department focuses on its own deliverables and deadlines rather than improving cross-cutting efficiency. As a result, there are few functions explicitly tasked with making *other teams* more productive, which leads to siloed efforts and duplicated work. It's easy to label systems as “inefficient,” but those inefficiencies usually exist *for a reason*. They arise from the way companies are designed and measured – for example, rewarding quick wins in individual silos rather than holistic improvements in workflow.

A classic illustration is how non-technical business teams adopt their own ad-hoc tools to meet pressing needs. When the official IT development pipeline is slow or under-resourced, **smart, well-intentioned employees** will find a way to solve their immediate problems, even if it means creating unofficial solutions ¹. This isn't malicious – it's a rational response to backlogs and pressure. As one industry commentary puts it, *“shadow IT doesn't start with bad intentions. It starts with backlogs... Business doesn't wait. So people...solve problems the only way they know how: with Excel”* ¹. In other words, workers optimize for getting the job done using the tools at their disposal, even if those tools are not ideal in the long run.

Understanding this economic reality – that individuals and teams optimize for their own short-term goals under given constraints – helps explain why certain “inefficient” practices flourish. Rather than simply blaming users for doing things the “wrong” way, we should examine **the reward structures and system design** that lead them there. Often, the enterprise has not provided an easy, sanctioned way to achieve a task *with* proper engineering and security considerations, so people turn to workarounds that maximize their immediate productivity.

Subpar Internal Tech and the Rise of Shadow IT

Because many organizations do not treat internal software development and tooling as a core competency, **the technology used by non-engineering departments is often subpar**. These business teams (HR, finance, operations, etc.) frequently rely on a patchwork of legacy systems, niche SaaS products, and manual processes. Purchasing decisions for internal tools might be made by people who aren't deeply technical – focusing on feature checklists and short-term needs while overlooking *non-functional requirements* like data portability, security, and maintainability. This can lead to heavy vendor lock-in (data trapped in proprietary formats) and fragmented data across many siloed apps. The overall landscape becomes a *mess of “accidental architectures”* driven by immediate needs rather than a cohesive tech strategy.

Spreadsheets are the poster child of this phenomenon. Excel and similar tools became ubiquitous in enterprises because they empower users to solve problems **without waiting for IT**. In fact, Excel has

been called “*the most ubiquitous programming environment on Earth*” ². Business users leverage spreadsheets to automate calculations, create reports, and prototype new ideas with an **instant feedback loop** – all on their own machine. The spreadsheet’s flexibility and approachability mean a finance or operations person can iterate on a model or process rapidly, **without formal software engineering skills**. Crucially, they can do so *without feeling judged* or slowed down by bureaucracy, which encourages innovation and learning in those early stages.

However, what begins as a quick fix in a spreadsheet can quietly grow into a critical piece of enterprise logic – essentially a **shadow IT system**. The one-off spreadsheet for budgeting or analysis accretes more data and complexity over time: it pulls data from other systems via copy-paste or scripts, sprouts complex formulas and macros, and might even function as an ersatz database ³. Soon, you have a workbook with 27 tabs and file names like `Final_v3_UPDATED_THIS_ONE.xlsx` that only a few “spreadsheet whisperers” understand ⁴. Mission-critical decisions end up relying on a fragile file sitting on someone’s desktop. As the Coherent Global team humorously described, the spreadsheet becomes “*that model*” — *a totem of both brilliance and fragility* ⁵. This is short-term ingenuity turning into long-term technical debt.

The **risks of this pattern** are well documented. Such unofficial tools often lack all the non-functional qualities that professional IT solutions would have. There’s no rigorous testing or change control, so one misplaced formula or accidental deletion can wipe out months of work (breakage risk) ⁶. Data in these user-managed tools is opaque to central IT, so it can’t be properly secured or backed up (creating blind spots for security) ⁶. And a solution that worked for 100 data entries may collapse under 10,000 (scalability issues) ⁷. In short, “*what worked for 100 quotes is a mess at 10,000*” ⁶. Shadow IT solutions typically don’t fail dramatically all at once; instead, they create **continuous drag and hidden vulnerabilities** that make everything harder to maintain ⁸.

Why do such suboptimal approaches persist? As noted, it’s often because the *alternative – getting a proper system built – is too slow or costly in the short run*. If requesting a new feature from IT means waiting a year, an analyst will find a quicker way today using a spreadsheet or an unsanctioned SaaS app. The underlying economic insight is that **each team is trying to maximize its own output**, and if the official process makes that difficult, unofficial methods fill the gap. This is the same reason we see “*shadow AI*” emerging now: when employees discover a powerful new technology like a cloud AI service that boosts their productivity, they might use it without approval if the official channels are cumbersome. (In fact, analysts have begun calling this *Shadow AI*, akin to shadow IT – where staff paste sensitive data into public chatbots because it helps them get work done, even though IT hasn’t vetted it ⁹.) The drive for immediate results often trumps latent security or governance concerns, until something goes wrong.

From Spreadsheets to “Vibe Coding”: Empowering Users in New Ways

The next evolution of this dynamic is now unfolding with the rise of “**vibe-coded**” applications. *Vibe coding* is a term coined to describe building software by interacting with AI/LLM tools in a conversational or intuitive way – essentially, using AI assistance to create custom apps on the fly based on one’s “*vibe*” or intent, rather than writing detailed code. In practical terms, it means a domain expert can spin up a mini-application by prompting an AI, without traditional programming. These *vibe-coded apps* are being heralded as “*the new spreadsheets*” because they let end users craft bespoke solutions quickly, just as spreadsheets did in the past, but with even greater power.

To understand why vibe coding is catching on, consider what makes spreadsheets so enduring: **immediate control and feedback**. A manager can lay out calculations in Excel and instantly see results, tweaking formulas on the fly. Vibe coding offers a similar empowerment, but goes further by handling complex logic, integration, and even UI generation via AI. As one AI professional demonstrated, we are *“rapidly approaching a reality where if the software you need doesn’t exist, you can simply will it into existence over a morning coffee.”* ¹⁰ In a recent example, a creator built a custom web-based spreadsheet-like tool from scratch in just **two hours** using an LLM (Google’s Gemini model) and dubbed this approach “vibe coding” ¹¹. The resulting prototype combined the familiarity of a grid interface with the generative power of AI – allowing the user to enter natural language commands and even generate or edit images within cells ¹² ¹³. This kind of outcome would have been unthinkable for a lone non-developer to achieve so quickly before the advent of advanced AI assistants.

The **appeal of vibe coding** is that it meets users where their needs are, solving very specific, niche problems that off-the-shelf software often ignores ¹⁴. Instead of contorting a generic tool to fit a unique workflow, a user can now create a custom tool that perfectly suits that workflow in a short time ¹⁵. In the spreadsheet era, if you found Excel’s capabilities lacking, you had to either live with it or ask IT for a new app. Today, if a product manager or HR analyst finds gaps in their software, they might *prompt an AI to create a tailored application or automation* just for them. The software adapts to the user, not the other way around ¹⁵. This is a profound shift in how technology can be delivered inside organizations – one that greatly accelerates innovation and experimentation. We can think of these AI-assisted mini-apps as the new **“macros on steroids.”** They go beyond the old Excel macros by tapping into extensive AI capabilities (like language understanding and image generation) and offering better user interfaces than a clunky spreadsheet.

However, just as spreadsheets came with **trade-offs and hidden costs**, vibe-coded apps bring their own set of challenges. By bypassing traditional development, they often bypass the safeguards and quality controls that come with it. The two-hour AI-built spreadsheet app mentioned above was explicitly *not* production-grade – the creator admitted it was *“not a replacement for fully robust software like Excel just yet”* ¹⁶. A quickly vibe-coded solution **“likely lacks the security features, massive data scalability, and collaboration reliability of enterprise tools,”** as the author noted ¹⁶. In other words, these AI-generated apps are powerful prototypes or stopgaps, but they may not meet the standards required for mission-critical use. Just as an Excel macro can unexpectedly break or produce errors, an LLM-driven workflow can behave in unpredictable ways because the AI doesn’t follow strict, deterministic logic. In fact, one fundamental difference is that **traditional software is predictable (the same input yields the same output every time), whereas an LLM’s output can vary and even be outright incorrect or bizarre** ¹⁷. This unpredictability is sometimes charming in a creative tool, but it’s dangerous in a business process. An AI agent wired into your systems might act like a *“Gremlin in the machine,”* doing things you didn’t anticipate – a bit of Dr. Jekyll and Mr. Hyde, if you will. As security researchers warn, over-reliance on such AI without proper checks *“creates fragility in business processes”* – when the model misfires or goes down, operations could grind to a halt ¹⁸.

Another risk is the **lack of oversight**. With spreadsheets, at least the logic was visible (if convoluted) to anyone who opened the file. With an AI-driven app, the decision-making might be happening inside an opaque model. Without careful design, you might not have an audit trail of why the AI did X or on what data it based a crucial decision. This is why many generative AI projects in enterprises stall after the proof-of-concept stage – Gartner predicts 30% will be abandoned by end of 2025 – often because companies realize they lack the risk controls and clear value proposition once the initial cool demo is done ¹⁹. The enthusiasm from quick wins can quickly meet the reality of compliance requirements, data governance, and reliability concerns.

In summary, vibe-coded applications are unleashing a new wave of **citizen development**, analogous to what spreadsheets did decades ago, but now supercharged by AI. They empower business users to create and iterate fast, targeting exactly their problem domain. This “wild west” of user-created AI tools is already forming the next layer of shadow IT (or “shadow AI” ⁹). Companies will soon have myriad little AI-infused tools running in departments under the radar – each solving a local problem, each potentially introducing new vulnerabilities.

Wardley Mapping the Evolution: From Genesis to Commodity

To make sense of the transition from spreadsheets to vibe-coded apps (and eventually to fully engineered solutions), it helps to use **Wardley Maps**. Wardley Mapping is a strategic tool that charts components of a system along two axes: the value chain (how visible a component is to the end user’s needs) and evolution (how mature or commoditized a component is, from novel “Genesis” to ubiquitous “Commodity”). By mapping out a user need and the components that fulfill it, we can visualize where we are *over-relying on custom, fragile elements* and where more mature, stable solutions exist.

Consider the example of a business user who has a need: say, “Generate a quarterly budget report with analysis.” In a Wardley Map, this **user need** would be at the top. Now consider how that need is met in three scenarios:

- **Scenario A: Spreadsheet DIY.** The user gathers data from various sources manually (perhaps exporting from finance systems) – that **data gathering** is a custom, ad-hoc step (far left on the evolution axis, since it’s done manually or via one-off scripts). They then use a spreadsheet (Excel, for instance) as the platform to calculate and analyze – the **spreadsheet software** itself is a commodity product (far right on the axis; Excel is standard and widely available). However, the actual **budget logic and formulas** in the spreadsheet are a *custom-built artifact* created by the user (left-of-center on the map). This logic is in a *Genesis/Custom* stage: unique to that user, not well-tested, possibly very innovative but also prone to error. The **security and controls** around this process (access control, data integrity checks) are minimal – perhaps just file permissions on the spreadsheet – which again is a very primitive level of maturity. If you mapped this out, you’d see critical components (data handling, logic, security) sitting on the left as custom/chaotic elements, with only the spreadsheet software and maybe the computer hardware on the right as commoditized components. The Wardley Map here highlights a big **gap**: important functions like data storage, version control, and security are in an immature state (handled manually or not at all). This is why the solution, while meeting the need in the short term, is fragile.
- **Scenario B: Vibe-Coded App.** Now imagine the user tries a more advanced route: they use a generative AI to create a small web application that automates the budget report. Perhaps they prompt an LLM to build a tool that pulls data via APIs, runs some analysis, and presents results in a nice interface. On the Wardley Map, the **UI and experience** might look more evolved (the interface could be as polished as a typical web app, leveraging modern frameworks – which are products/commodities). The **underlying services** like a database or an API might also be standard components if the AI hooked into common platforms. Superficially, it appears we’ve moved rightward (toward more evolved components) compared to the spreadsheet. *But* a critical piece – the **decision-making logic** – is now powered by an LLM, which in terms of behavior is **more unpredictable and novel** than even the user’s spreadsheet formulas were. LLM behavior is at the cutting edge (arguably a *Genesis* phase component in the context of enterprise IT) because it’s non-deterministic and still evolving rapidly. We’ve essentially replaced the human’s ad-hoc logic with an AI’s ad-hoc logic. So on the map, some parts of the system have shifted right (using commodity cloud services perhaps), but the core analytical engine might have *moved*

leftward in maturity (from a human's relatively common-sense judgment to an AI's still-experimental reasoning). **Wardley Maps can illustrate this paradox:** an AI-generated app might use shiny modern components (giving a false sense of maturity), yet embed a highly novel component at its core (the LLM's "brain" making decisions).

- **Scenario C: Engineering Team Handoff.** In the third scenario, the organization takes either the spreadsheet (A) or the vibe-code prototype (B) and hands it to a proper engineering team to **industrialize** it. Now the Wardley Map for fulfilling that user need shifts dramatically to the right. The **data** is pulled through well-defined pipelines or APIs (custom-built or productized integrations, but managed with standard practices). The **business logic** is implemented in a more deterministic way – perhaps coded in a proper application with tests, or using the LLM but with guardrails and monitoring. The **security, version control, and reliability features** are designed and implemented as first-class concerns (using established frameworks, libraries, and practices – all commodity or product-level components). Essentially, the parts of the value chain that were ad-hoc get refined and moved to a more evolved state. On a Wardley Map, more components now sit on the right side: the system becomes composed of stable products or well-understood utilities (e.g. a secure database, an audited codebase, standardized identity management for access). The user still gets their budget report, and maybe still via a familiar interface (perhaps even an Excel front-end or a simple web form), but behind that interface there is now a **well-governed, scalable engine** ²⁰. This is the ideal end-state: retain the *speed and flexibility* at the front-end while adding the *safety, security, and scalability* at the back-end.

Using Wardley Maps in this context helps communicate a few key points to stakeholders:

- **Innovation starts on the left.** It's natural that new solutions begin in a chaotic, experimental form (like a skunkworks spreadsheet or a vibe-coded script). This is the **Genesis** phase where a novel approach is tried.
- **Handover is critical to evolve right.** At some point, if the solution proves its value, you must invest in moving it rightward along the evolution axis – i.e. **hand it over to an engineering process** that can turn it into something more like a Product/Commodity in its internal components. If you don't, the map shows you're depending on a lot of custom, fragile pieces for a high-value need – a recipe for future failure.
- **Not everything should be a product from day one.** We shouldn't mandate that every idea go through IT from the start – that would stifle innovation at genesis. Instead, encourage experimentation but with a pathway to **"industrialize" successful experiments**. Wardley Maps visually reinforce that concept of *Pioneers -> Settlers -> Town Planners* (a Wardley analogy): the pioneers (end users with spreadsheets or prompts) discover new needs and solutions, then settlers (engineering teams) turn those into more solid custom-built applications, and finally town planners (platform teams) commoditize the capability for everyone. In our example, an engineer might take the vibe-coded app and rebuild its core as a robust service, and later the IT department might offer a self-service budgeting tool to all, making the custom app obsolete.

For the purposes of the presentation, we could animate such a map showing the journey of a capability (budget analysis, or any other business process) from left to right over time – starting in a spreadsheet, evolving through a vibe-coded AI tool, and ending as a proper enterprise application. This visual can drive home why simply allowing chaotic innovation (left side) is not enough – and conversely, why expecting everything to be fully commoditized (right side) isn't realistic either. **The organization needs a pipeline to cultivate and mature these solutions.**

Embracing and Managing the “Vibe” Revolution

The advent of easy AI-driven development tools is a game-changer for businesses, and it’s one that **IT and security teams must proactively embrace and manage** rather than resist. The genie is out of the bottle: just as spreadsheets could not be banned from offices (they were too useful to too many people), we won’t be able to prevent eager employees from using AI copilots to build mini-apps that make their jobs easier. Attempting to shut this down would only drive it further into the shadows. Instead, the smarter approach is to **provide safe frameworks and supportive oversight** for this new wave of citizen development.

Firstly, organizations should **pair business “vibe coders” with engineering buddies or teams**. The goal is to create a partnership model: when someone in a department comes up with a brilliant little automation or application that starts getting popular, there is a mechanism to bring in software engineers who can fortify it “under the hood.” Think of it as catching a falling prototype and gradually refactoring its innards without losing its outward utility. This doesn’t mean smothering the innovation – it means nurturing it. By designing a handover process (or even tools that allow export of a prototype into a developer-readable format), companies can capture the best of both worlds: **rapid innovation and eventual rigor**. In practice, for example, if a financial analyst vibe-codes a tool that many people start relying on, the IT team could step in to integrate that tool with official data sources, add error handling, implement authentication, and so on, while keeping the user interface and workflow that the analyst designed. The result is a more reliable solution that still precisely fits the business need.

Security teams have a golden opportunity here to reposition themselves as **enablers of innovation** rather than gatekeepers. Historically, security is seen as the department of “no,” throwing a wrench into fast-moving projects due to compliance concerns. But with shadow AI and vibe coding on the rise, security can instead offer to be the adults in the room who make the cool new toy safe for everyone to play with. For instance, security professionals can develop guidelines and libraries for prompt engineering that help avoid known pitfalls (like prompt injections or data leaks). They can set up monitoring for these AI-driven apps to detect misuse or anomalies (treating an LLM somewhat like one would monitor an API, since, as noted, **LLMs blur the line between code and data in ways traditional security tools don’t cover** ²¹). Security can also help classify what kind of data is safe to feed into an unsanctioned AI service and what isn’t, to prevent those data leakage nightmares where employees paste confidential info into ChatGPT. In fact, encouraging the use of *approved* internal LLM platforms (with logging, data protection, etc.) is far better than ignoring the issue and having an uncontrolled “Shadow AI” situation ⁹.

From a risk perspective, security teams should conduct **risk assessments of these vibe-coded workflows** early. Rather than waiting for an incident, they can proactively map out possible failure modes: What if the LLM gives a wrong recommendation that someone acts on? What if the prompt logic is manipulated (prompt injection) to produce harmful actions? What if the mini-app connects to other systems with too many privileges? By asking these questions, security can advise on where a human-in-the-loop is needed, or where additional approval should be built in, or where the AI’s autonomy should be limited. This approach aligns with emerging best practices for AI governance which suggest treating AI systems as **“living” components that require continuous oversight, testing, and tuning** ²² ²³ – much like one would continuously patch and monitor a critical software service.

There is also a broader **strategic angle**: The security team’s remit can evolve to include safeguarding the *process of innovation* itself. This means championing the idea that building technology (even if just internal tools) is a core business capability and should be done responsibly. If the security group can help develop a framework where non-engineers can build small apps safely (with guardrails like

sandboxed environments, audit logs, and easy escalation to IT when needed), they become the heroes who enabled the business to move faster *securely*. This is a far more valuable role than simply being the compliance police for traditional systems. In fact, if security teams fail to adapt, they risk irrelevance: other parts of the organization (or new specialized teams) might take over the function of managing AI tools and internal development, leaving the old-school security folks sidelined. We've seen analogous shifts in areas like cloud DevOps – where operations and security had to reinvent themselves (DevSecOps) or get left behind.

To summarize this part: **vibe-coded applications are coming whether we like it or not**. They will proliferate because they address a genuine gap – the need for speed and personalization in software. The task for IT and security is to **embrace this trend and steer it**. Provide the platforms, governance, and collaborative culture so that when a business user comes up with the next brilliant mini-app, the organization can capture that value *safely and at scale*. This includes funding the maturation of these apps (the business must be willing to invest in “productizing” their spreadsheet or AI hack once it proves useful) – essentially budgeting for those non-functional requirements that were initially skipped. The business side may initially balk at spending time and money on security, stability, refactoring, etc., but as they experience the brittleness of unrefined prototypes, they'll understand why it's necessary. It's our job in security/IT to articulate that in terms of **risk and reward**: “Yes, your team saved 3 months by vibe-coding this tool – now if you want to rely on it for the next 3 years, here's what we need to do to make sure it doesn't *blow up* at the worst time.”

In conclusion, the themes of *Part 4* tie together the economics of internal innovation and the advent of AI-driven development. We see that many inefficiencies are born from rational behavior under constraints, and technologies like spreadsheets and now LLMs provide outlets for that behavior. Our challenge (and opportunity) is to **acknowledge those forces** – not dismiss them – and build a bridge between rapid, user-driven innovation and the robustness of engineered, secure systems. By using tools like Wardley Maps to visualize the journey and by proactively engaging with the vibe-code movement, we can turn a potential chaos into a strategic advantage. The companies that manage to do this will not only reduce shadow IT risks but actually harness the creativity of their people, with security and IT as co-creators of value. That, ultimately, is a far more **economically efficient outcome** than the status quo, turning what could be a perfect storm of uncontrolled tech into a well-navigated voyage toward competitive advantage.

1 16

1 2 3 4 5 6 7 8 20 Running on Excel and Prayers: Signs You've Built a Shadow IT System

<https://www.coherent.global/blog/running-on-excel-and-prayers-signs-youve-built-a-shadow-it-system>

9 18 19 21 22 23 LLM Risks: Enterprise Threats and How to Secure Them

<https://www.lasso.security/blog/llm-risks-enterprise-threats>

10 11 12 13 14 15 16 Vibe Coding: Build an AI Spreadsheet in Just 2 Hours

<https://cybercorsairs.com/vibe-coding-a-spreadsheet-in-2-hours-with-ai/>

17 What Are LLM Security Risks? And How to Mitigate Them

<https://www.sentinelone.com/cybersecurity-101/data-and-ai/llm-security-risks/>